

Bases de Datos

Recuperación de Fallas — Redo, Undo/Redo y WAL

Recordatorio de la clase anterior

- El **Log Manager** registra toda la actividad de las transacciones en un archivo secuencial.
- **Undo Logging**: el log contiene **valores antiguos** (`<T, X, t_old>`); en recovery, **deshacemos** las transacciones incompletas.
- Reglas U1 / U2: log primero, datos después, COMMIT al final.
- **Checkpoints no-quiescentes** acotan cuánto del log hay que leer.

Problema que dejamos abierto: Undo Logging **obliga a flush** los datos a disco antes de hacer COMMIT → los commits son lentos.

Agenda

1. **Redo Logging**: confirmar primero, escribir datos después.
2. **Undo/Redo Logging**: lo mejor de ambos mundos.
3. Comparación de las tres estrategias.
4. **Write-Ahead Logging (WAL)** en PostgreSQL.
5. Cierre del bloque de transacciones.

Redo Logging

Idea

No escribimos los datos a disco antes de confirmar. Solo escribimos al log.

Si la transacción se confirmó pero los datos aún no llegaron a disco, en recovery los **re-hacemos**.

Para re-hacer una escritura sobre X, necesitamos guardar el **valor nuevo**.

Registro	Significado
<START T>	T inició
<COMMIT T>	T confirmó
<ABORT T>	T fue cancelada
<T, X, v>	T modificó X; v es el valor nuevo

La regla de Redo

R1: antes de que cualquier modificación de T llegue a disco, **todos** los registros de log de T — incluido **<COMMIT T>** — deben estar en disco.

```
escribir log <T, X, v>
      ↓
escribir <COMMIT T> en log
      ↓
flush del log a disco
      ↓
escribir X (modificado) en disco (cuando convenga)
```

Inversión respecto a Undo: ahora el log con COMMIT está en disco **antes** que los datos. El commit es **rápido**: solo log.

¿Por qué funciona?

Imaginemos un crash:

- **Antes del flush del log con `<COMMIT T>`** : T no aparece como committed → la consideramos no realizada. Sus modificaciones tampoco están en disco (regla R1).
- **Después del flush del log con `<COMMIT T>` , pero antes de escribir X a disco:** sabemos por el log que T se commiteó y cuál es el valor nuevo de X → **re-hacemos** la escritura.
- **Después de escribir X a disco:** ya estaba bien; re-aplicar es **idempotente**.

Algoritmo de recovery (Redo)

Recorrer el log **del inicio hacia el final** (la dirección opuesta a undo).

```
Confirmadas = { T : <COMMIT T> aparece en el log }
```

```
para cada registro r del log, en orden hacia adelante:
```

```
    si r = <T, X, v> y T ∈ Confirmadas:
```

```
        escribir X := v en disco (re-hacer)
```

```
    si r = <T, X, v> y T ∉ Confirmadas:
```

```
        ignorar (T no se commiteó → no la queremos)
```

```
para cada T que abrió START pero no tiene COMMIT:
```

```
    escribir <ABORT T> al final del log
```

Idempotente: correr recovery dos veces da el mismo resultado.

Ejemplo

Log antes del crash:

```
<START T1>
<T1, A, 7>           ← A pasó a valer 7
<START T2>
<T2, B, 11>
<COMMIT T1>
<T2, C, 15>
x CRASH x
```

Recovery (de inicio a fin):

- T1 está en **Confirmadas**, T2 no.
- **<T1, A, 7>** : T1 confirmada → **A := 7**.
- **<T2, B, 11>** : T2 no confirmada → ignorar.
- **<T2, C, 15>** : T2 no confirmada → ignorar.
- Escribir **<ABORT T2>** al final.

Checkpoints en Redo Logging

1. Escribir <START CKPT (T1, ..., Tn)> ← T's activas (sin COMMIT aún)
2. Hacer flush a disco de todas las modificaciones de transacciones que ya están commiteadas hasta ese punto
3. Escribir <END CKPT>

Diferencia clave con Undo: acá flusheamos solo lo de las transacciones **ya confirmadas**, no lo de las activas.

Recovery con checkpoint (Redo)

Recorrer el log **hacia atrás** buscando `<END CKPT>` y su `<START CKPT (T1, ..., Tn)>` :

- Todas las transacciones que se commitearon **antes** de `<START CKPT>` ya tienen sus datos en disco → **no** necesitan redo.
- Para las transacciones con commit **después** de `<START CKPT>` (incluyendo las T1..Tn si commitearon antes del END), aplicar redo desde el inicio de la **más antigua** entre T1..Tn.

Si encontramos `<START CKPT>` sin su `<END CKPT>` → retroceder al checkpoint anterior completo.

Ejemplo

```
<START T1>  
<T1, a, 5>  
<START T2>  
<COMMIT T1>  
<T2, b, 10>  
<START CKPT (T2)>  
<T2, c, 15>  
<START T3>  
<T3, e, 25>  
<END CKPT>  
<COMMIT T2>  
<COMMIT T3>  
x CRASH x
```

Recovery: retrocedo, encuentro `<END CKPT>`, busco `<START CKPT (T2)>`. T2 es la más antigua activa al inicio del checkpoint. Redo desde **el inicio de T2**: aplicar `b:=10`, `c:=15`, `e:=25`. T1 commiteó antes del checkpoint → ya está en disco. Listo.

El costo de Redo

- Las modificaciones se **acumulan en el buffer pool** sin escribirse a disco hasta el flush del checkpoint.
- Si hay muchas transacciones largas, el buffer se **satura** con páginas sucias.
- Eventualmente hay que escribir igual → la presión sobre el buffer manager aumenta.

→ **Trade-off opuesto al de Undo**: ahora los commits son rápidos, pero el manejo de páginas sucias en RAM es más complejo.

Undo/Redo Logging

Idea

Guardar ambos valores (viejo y nuevo) en cada registro de update.

Registro	Significado
<code><T, X, t, v></code>	T modificó X; t valor antiguo, v valor nuevo

Flexibilidad: ya no necesitamos forzar el orden estricto entre escribir datos y escribir `<COMMIT T>`.

Reglas:

- **UR1 (WAL):** el log `<T, X, t, v>` se escribe **antes** que cualquier copia de X a disco.
- **No** hay restricción sobre el orden relativo entre escribir X a disco y escribir `<COMMIT T>` al log.

Algoritmo de recovery (Undo/Redo)

```
Confirmadas = { T : <COMMIT T> aparece en el log }
```

```
(1) Redo hacia adelante:  
para cada r en orden hacia adelante:  
    si r = <T, X, t, v> y T ∈ Confirmadas:  
        escribir X := v en disco
```

```
(2) Undo hacia atrás:  
para cada r en orden inverso:  
    si r = <T, X, t, v> y T ∉ Confirmadas:  
        escribir X := t en disco
```

```
para cada T sin COMMIT ni ABORT:  
    escribir <ABORT T> al final del log
```

¿Por qué dos pasadas? Una T confirmada puede tener escrituras pendientes (→ redo); una T no confirmada puede tener escrituras ya en disco (→ undo). Cada pasada cubre uno de los casos, y es idempotente.

Checkpoints en Undo/Redo

1. Escribir <START CKPT (T1, ..., Tn)> $\leftarrow$ T's activas
2. Flushear a disco todas las páginas sucias del buffer pool
3. Escribir <END CKPT>

Más relajado que Undo y Redo:

- En **Undo** había que **esperar** a que las activas terminaran para poder flushearlas con seguridad.
- En **Redo** flusheábamos **solo las confirmadas** (no podíamos deshacer una activa si después abortaba).
- En **Undo/Redo** flusheamos **lo que sea**, sin esperar: el log tiene ambos valores.

Checkpoints en Undo/Redo

1. Escribir `<START CKPT (T1, ..., Tn)>` ← T's activas
2. Flushear a disco todas las páginas sucias del buffer pool
3. Escribir `<END CKPT>`

Recovery con checkpoint: retrocedo hasta `<END CKPT>` y su `<START CKPT (T1..Tn)>` .
El redo arranca desde el inicio de la T más antigua entre T1..Tn; el undo cubre las incompletas en ese rango.

ARIES (IBM, 1992)

este es el sistema que implementan DBMS actuales. No se escriben valores antiguos ni nuevos en el log, sino datos físicos: en qué parte del disco se escribe.

Tres pasadas sobre el log:

1. **Analysis** — desde el último checkpoint hacia adelante: reconstruye la *Transaction Table* (T's activas al crash) y la *Dirty Page Table* (páginas con cambios posiblemente no flushados a disco).
2. **Redo** — desde el LSN sucio más antiguo: re-aplica **todo**, incluso de transacciones que después abortaron. La BD queda exactamente como justo antes del crash.
3. **Undo** — hacia atrás, solo para las T's de la Transaction Table.

Idempotencia: cada operación de recovery se logea en el **CLR** (Compensation Log Record). Si crasheamos durante recovery, los CLR's nos dicen hasta dónde llegamos. Así el undo también es idempotente.

Comparación de estrategias

Aspecto	Undo	Redo	Undo/Redo
Contenido del log de update	valor viejo	valor nuevo	ambos
Antes de <code><COMMIT T></code> , los datos están...	en disco	no necesariamente	no necesariamente
Recovery: trans. incompletas	deshacer	nada que hacer*	deshacer
Recovery: trans. commiteadas	ignorar	rehacer	rehacer
Costo en commit	flush datos + log	solo log	solo log
Costo en buffer	bajo	alto (datos sucios)	medio

WAL: el log como fuente de verdad

Reencuadre

Hemos hablado del log como **herramienta de recovery**. Pero en sistemas modernos, ese mismo log es:

- **La fuente de durabilidad** del DBMS — vimos esto.
- **El protocolo de replicación** — las réplicas reciben WAL records y los aplican.
- **El cable de integración** del stack de datos.
- **El sustrato de almacenamiento** de algunos sistemas — Donde directamente *guardan* el WAL y reconstruyen páginas a demanda.

"The log is the database." El recovery es solo el primer uso.

Write-Ahead Logging (WAL)

Principio universal: *cualquier modificación a una página de datos debe estar registrada en el log — y el log debe estar persistente en disco — antes de que la modificación misma llegue a disco.*

Es la regla común a Undo, Redo y Undo/Redo.

- PostgreSQL, MySQL, Oracle, SQL Server: **todos** usan WAL.
- En PostgreSQL los registros se llaman **WAL records**, viven en archivos `pg_wal/...`, y se segmentan en archivos de 16 MB cada uno.

Ciclo de vida de una transacción en PostgreSQL

BEGIN;

UPDATE cuentas **SET** saldo = saldo - 100 **WHERE** cid = 1;

↓

(1) modifica la página en buffer pool, marca dirty

(2) anota un registro WAL en el buffer WAL

UPDATE cuentas **SET** saldo = saldo + 100 **WHERE** cid = 2;

↓

(1)(2) lo mismo

COMMIT;

↓

(3) escribe <COMMIT T> en el buffer WAL

(4) fsync del WAL a disco ← acá termina realmente el commit

(5) responde "OK" al cliente

Ciclo de vida de una transacción en PostgreSQL

Las páginas sucias del buffer pool **no** se flushean en el commit. Las flushea:

- El **bgwriter** (background writer) continuamente, manteniendo páginas limpias para reciclar.
- El **checkpointer**, en cada checkpoint, para acotar el WAL que recovery necesita leer.

PostgreSQL = WAL + MVCC (no undo log)

Una distinción importante: PostgreSQL **no mantiene un undo log separado**.

- Cuando una transacción modifica una fila, PG crea una **nueva versión** de la tupla — la vieja sigue ahí, marcada con la xid que la creó.
- Si la transacción **commitea**, la nueva versión se vuelve visible (y la vieja queda en estado "muerto").
- Si la transacción **aborta**, simplemente nunca se hace visible — no hay que "deshacer" nada.

→ El undo es **gratis** en PG: lo provee el modelo de versiones (MVCC).

→ El WAL es **solo redo** en PG: registra el cambio que llevó a la nueva versión.

→ Un proceso de **vacuum** se encarga de eliminar las versiones muertas en background.

Checkpoints en PostgreSQL

- Disparados cada `checkpoint_timeout` (5 min por defecto) o cuando el WAL llega a `max_wal_size`.
- Son **no-quiescentes**: la BD sigue aceptando transacciones.
- Flushean páginas sucias del buffer pool a disco y escriben un *checkpoint record* en el WAL.
- En recovery, postgres busca el último checkpoint y empieza a aplicar redo desde ahí.

```
postgres=# SHOW checkpoint_timeout;  
postgres=# SHOW max_wal_size;  
postgres=# SELECT * FROM pg_stat_bgwriter;
```

Recovery en la práctica

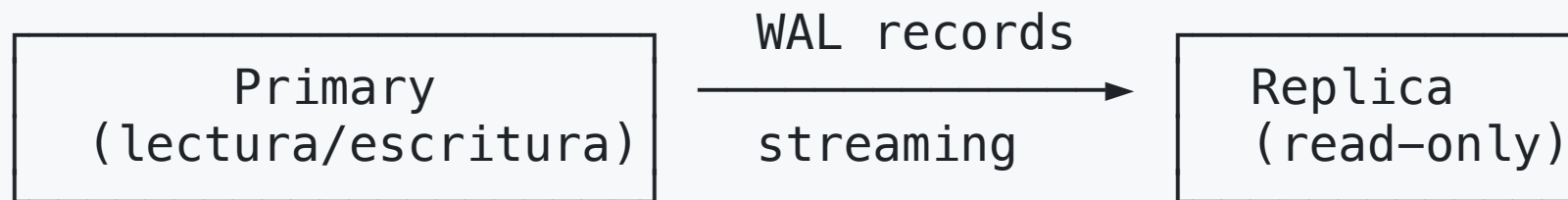
Si PostgreSQL detecta al iniciar que la última detención fue inesperada (crash):

```
LOG:  database system was interrupted; last known up at ...  
LOG:  database system was not properly shut down; automatic recovery in progress  
LOG:  redo starts at <LSN>  
LOG:  redo done at <LSN>, elapsed time: ... s  
LOG:  database system is ready to accept connections
```

→ Lee el WAL desde el último checkpoint, aplica redo. El undo de transacciones no commiteadas es implícito: sus versiones de tupla simplemente nunca se vuelven visibles.

El log como mecanismo de replicación

En PostgreSQL el mismo WAL que protege contra crashes es lo que se envía a las réplicas:



- **Streaming replication:** la réplica aplica los WAL records en tiempo real (`pg_wal_replay_lag`).
- **Hot standby:** la réplica acepta lecturas mientras replica.
- **Point-in-time recovery (PITR):** restaurar un backup + replay del WAL hasta un instante específico → volver al estado de "ayer 3pm antes del DROP TABLE".

Change Data Capture (CDC): el WAL como cable de integración

El WAL no es solo para réplicas internas. Las herramientas modernas **leen el WAL** para llevar los cambios fuera del DBMS:



→ Una **UPDATE** en el OLTP termina, segundos después, como una fila nueva en el data warehouse — sin tocar la app.

Esto es parte de la transacción - como un trigger.

Cierre del bloque ácido

Garantía ACID	Mecanismo
Atomicity	Log + recovery (undo de incompletas)
Consistency	Restricciones declarativas + lógica del programador
Isolation	2PL / niveles de aislamiento
Durability	log + recovery

